Skill-Portfolio

대표 기술
임베디드 펌웨어

개인 정보
성 명 : 곽혜민
E-mail : gtg1349@gmail.com

교육 이수사항		
교육명	**내용**	**교육기관**
웹서비스 개발자양성과정	**웹 서 비 스 기 획 , 구 현 및 배 포**	**강남구 혁신인재육성아카데미**
AI시스템 반도체 SW (2기)	**C 기반 임베디드 SW 개발 교육 이수 및 AVR·STM32 MCU 제어, Linux 시스템 프로그래밍, FPGA 설계, 팀 프로젝트 수행**	**대한상공회의소 서울기술교육센터**

전문설계 소프트웨어 능력	
언어(툴)	**활용능력**
C · AVR / STM32	Timer·외부 인터럽트·UART·PWM·DMA 레지스터 설정, ISR 역할 분리
C · Linux (Jetson)	I2C 센서 보정·LCD 4비트 출력, pthread·mutex 표시 스레드 분리
Verilog · Vivado	FSM·입력 동기화·디바운스 설계, 테스트벤치 타이밍 검증
Python · TensorRT	YOLO11n 미세조정, ONNX→TensorRT 변환·추론, NMS 후처리
C++ Qt · Flask	Qt5 시리얼 제어·카메라 스레드 GUI, Flask·SQLite 이력 조회

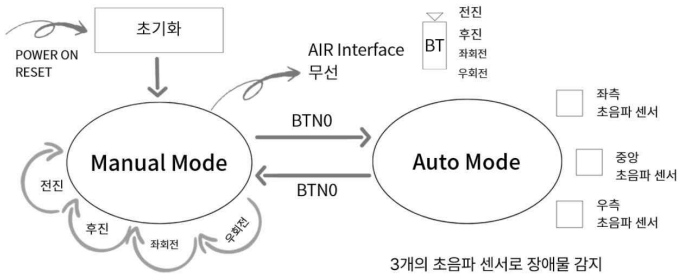
프로젝트 수행사항			
번호	**프로젝트명**	**프로젝트 내용**	**수행기관**
1	AVR 장애물 회피 RC카	초음파 3방향 자율주행·블루투스 조종	대한상공회의소 서울기술교육센터
2	STM32 베어메탈 제어	레지스터 기반 모터·DMA LED 제어	대한상공회의소 서울기술교육센터
3	AVR RTC·LCD 계산기	RTC 시계·괄호 사칙연산 계산기	대한상공회의소 서울기술교육센터
4	Jetson 스마트 선풍기	Linux I2C 센서·LCD 연동	대한상공회의소 서울기술교육센터
5	Basys3 I2C 센서 모니터	두 FPGA 간 I2C 센서값 전송·Qt 모니터	대한상공회의소 서울기술교육센터
6	STM32·Qt 차량 모니터링	Qt GUI 차량 제어·영상 모니터링	대한상공회의소 서울기술교육센터
7	Jetson 손동작 가위바위보	YOLO11n·TensorRT 손동작 인식 게임	대한상공회의소 서울기술교육센터
8	Board Guard PCB 검사	검사 대시보드·SQLite 이력 조회	대한상공회의소 서울기술교육센터

프로젝트 기술서

작 성 자
곽혜민

1	프로젝트명 : AVR 기반 장애물 회피 RC카
수행기간	2026.06.22 ~ 2026.06.24
담당역할	2인 프로젝트 · 펌웨어 전체 담당 담당 범위: 센서 측정 · 주행 판단 · PWM 구동 · UART 통신 · FND 표시
수행목표	수동·자동 주행을 하나의 MCU에서 처리하고, 주행 중에도 통신과 표시가 끊기지 않도록 자원 분리 ATmega128A 하나에서 블루투스 수동 조종과 초음파 자율 주행을 구현했습니다.
사용 기술	C · ATmega128A · 16MHz · Atmel Studio 7 · KiCad(회로도) 주요 구성: 초음파 센서 3개 · DC 모터 2개 · 블루투스 · 4자리 FND 2개 · 버튼
온라인 포트폴리오	https://hyehye12.github.io/projects/avr-auto-car

세부수행내용



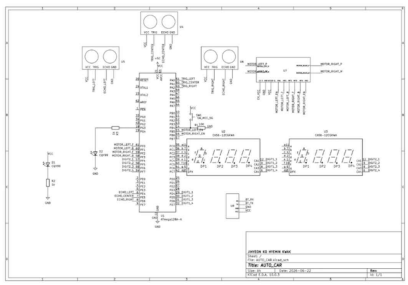


그림 1(왼쪽). 동작 모드. 전원 인가 후 수동 모드로 시작하며 BTN0으로 수동·자동을 전환합니다.

그림 2(오른쪽). KiCad 회로도. 초음파 Trigger는 PA0~2, Echo는 INT4~6(PE4~6), 모터 방향과 FND 자리 선택은 PORTF를 나눠 씁니다.

■ 거리 측정과 주행 판단

□ 타이머와 인터럽트 역할 분리

자원	담당 기능	구현 기준
Timer1	모터 PWM	64분주, TOP 1023(약 244Hz), 주행 상태에 따라 비교값 변경
Timer2	FND 다중화	1ms마다 표시 자리 갱신
Timer3 · INT4~6	Echo 펄스 폭 측정	64분주, 카운트당 4μs
UART1 RX 인터럽트	명령 수신	9600bps, 16바이트 링 버퍼에 저장 후 메인 루프에서 순서대로 처리

세부수행내용 (계속)

□ 초음파 측정 과정

측정 센서 선택 → Trigger 10μs 출력 → Echo 상승 감지
Echo 상승 시 Timer3 초기화 → 하강 시 카운트 확보 → cm 환산

좌·중·우 센서를 순서대로 측정하고 각 측정 뒤 20ms를 기다리며, 활성 센서의 Echo만 처리합니다. 거리(cm)는 Timer3 카운트 × 2 / 29(4μs 카운트 ÷ 58μs/cm)로 환산합니다. Trigger 후 5ms 안에 Echo 상승이 없으면 센서 실패(0), 상승 후 30ms 안에 하강이 없으면 범위 밖(400)으로 처리합니다.

```
ISR(INT4_vect) {                    // 왼쪽 센서 Echo
    if (active_sensor != SENSOR_LEFT) return;
    if (PINE & (1 << ECHO_LEFT)) {      // 상승 엣지
        TCNT3 = 0;
        rising_edge_received = 1;
    } else if (rising_edge_received) {  // 하강 엣지
        left_distance = ((uint32_t)TCNT3 * 2) / 29;
        rising_edge_received = 0;
        active_sensor = SENSOR_NONE;
        ultrasonic_done = 1;
    }
}
```

16비트 TCNT3는 두 번에 나눠 읽히므로, 메인은 ATOMIC_BLOCK으로 인터럽트를 잠시 막고 TCNT3와 측정 상태를 함께 읽습니다. 센서 하나의 최대 대기는 약 35ms입니다.

□ 자동 주행 조건

조건	명령	PWM 비교값	듀티(비교값/1024)
중앙 거리 = 0	정지	0	0%
0 < 중앙 거리 < 15cm	후진	150	14.6%
15 ≤ 중앙 거리 < 30cm, 좌·우 모두 0	정지	0	0%
15 ≤ 중앙 거리 < 30cm, 좌 > 우	좌회전	160	15.6%
15 ≤ 중앙 거리 < 30cm, 그 외	우회전	160	15.6%
중앙 거리 ≥ 30cm (범위 밖 400 포함)	전진	180	17.6%

□ 수동 모드 UART 명령

F/B(전진·후진), L/R(좌·우회전), S/E(정지·주행 종료) 명령을 대소문자 모두 처리합니다. 수신 ISR은 16바이트 링 버퍼에 저장만 하고 메인 루프가 순서대로 실행합니다(사후 점검에서 1바이트 변수 방식의 명령 유실을 발견해 변경).

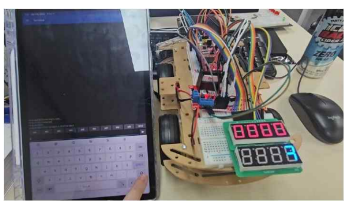


그림 3. 수동 모드 동작. 태블릿 블루투스 앱으로 명령을 보내고, FND에 주행 시간과 동작 상태를 표시합니다.

세부수행내용 (계속)

■ 문제 해결과 동작 확인

□ 문제 1. 고속 주행 중 거리값 변동과 코너 정체

현상	동일 코스 반복 주행 중, 고속에서는 첫 코너 모서리에 갇혀 후진과 잘못된 방향 회전을 반복
원인	판단 1회에 약 65ms가 걸리고 그동안 모터가 직전 명령을 유지해, 고속일수록 다음 판단 전에 모서리 깊숙이 진입
해결 · 확인	자동 모드 구동 속도를 낮춰 한 판단 주기 동안의 이동 거리를 줄임 → 동일 코스 전체 통과 확인
개선 방향	센서값 튐 → 최근 측정값 메디안 필터와 센서 간 대기 시간 단축 적용 예정

□ 문제 2. 공유 포트 갱신 중 표시 비트 보호

상황	PORTF 하위 4비트는 모터 방향, 상위 4비트는 FND 자리 선택에 사용
위험	방향 변경의 읽기·수정·쓰기 도중 1ms 표시 인터럽트가 상위 비트를 바꾸면 이전 값으로 덮어써질 수 있음
해결	마스크 연산과 임계 구역(SREG 저장 → cli() → 갱신 → SREG 복원) 적용

```
uint8_t sreg = SREG;
cli();
MOTOR_DIRECTION_PORT =
    (MOTOR_DIRECTION_PORT & ~MOTOR_DIR_MASK)
    | (dir_bits & MOTOR_DIR_MASK);
SREG = sreg;
```

MOTOR_DIR_MASK = 0x0F로 하위 4비트(모터 방향)만 바꿔 상위 FND 자리 선택 비트를 보존하고, 변경 전 인터럽트 상태를 복원합니다. (pwm.c set_direction)



그림 4. 자동 모드 주행 영상 캡처. 상자 코스에서 장애물을 피해 회전하며 진행하는 장면(시작 · 중간 · 끝)

□ FND 표시와 주행 결과 조회

Timer2 인터럽트가 1ms마다 한 자리씩 8자리를 순환 표시합니다. 주행 중에는 경과 시간(MMSS)과 F·B·L·R·STOP 상태를, E 명령으로 주행을 끝낸 뒤에는 버튼을 누를 때마다 누적 시간 → 후진 → 좌회전 → 우회전 횟수를 보여 주며 같은 동작의 연속 호출은 중복 집계하지 않습니다.

관련 코드: fnd.c (fnd_set_state, fnd_next_page)

□ 결과와 배운 점

수동 명령, 자동 회피, 모드 전환과 FND 상태 표시를 하나의 MCU에 통합했습니다. 회피 성능이 센서 처리뿐 아니라 차량 속도와 재판단 주기에도 좌우된다는 점을 확인했고, 포트 공유 기능의 비트 보존과 메인·ISR 공유 자원(16비트 타이머 레지스터, 수신 버퍼) 보호를 함께 고려했습니다.

프로젝트 기술서

작 성 자
곽혜민

2	프로젝트명 : STM32 베어메탈 모터 제어 · WS2812B Timer·DMA 제어
수행기간	2026.07.22 ~ 2026.07.26 (WS2812B 개인 실습: 2026.07.29)
담당역할	3인 프로젝트 · 인터럽트 구조 개편과 UART 속도 제어 담당 WS2812B Timer·DMA 제어: 개인 실습
수행목표	버튼 짧게 누름으로 방향 전환, 3초 길게 누름으로 정지, UART 숫자 명령으로 속도 변경 STM32F411RE에서 CMSIS와 레지스터 설정으로 EXTI·타이머·UART를 제어했습니다.
사용 기술	C · STM32F411RE · CMSIS · GCC Makefile 사용 자원: EXTI13(버튼) · TIM2 · TIM4 · TIM5 PWM · USART2 RX 인터럽트 / DMA1 Stream1
온라인 포트폴리오	모터 제어: https://hyehye12.github.io/projects/stm32-motor-control WS2812B: https://hyehye12.github.io/projects/stm32-ws2812b

세부수행내용

■ STM32 베어메탈 모터 제어

□ 입력 대기와 상태 처리 분리

버튼 해제를 기다리던 초기 구조를 이벤트 전달 방식으로 변경했습니다. ISR은 이벤트와 값을 저장만 하고 메인 루프가 상태를 전환해, 버튼을 누르는 중에도 타이머 만료와 UART 명령을 처리하도록 구성했습니다.

EXTI(누름·뗌) · TIM2/TIM4 만료 · USART2 수신 → ISR에서 이벤트·값 저장 (출력·상태 전환 없음)
메인 루프: 길게 누름 만료 → 누름·뗌 → 방향 전환 대기 만료 → UART 순서로 확인 후 상태 전환

입력 또는 자원	처리 내용
짧은 누름 / 방향 전환	STOP에서 구동, 회전 중에는 300ms 정지 후 반대 방향
TIM2 / TIM4	3초 길게 누름 판정 / 방향 전환 전 300ms 정지
UART / TIM5	숫자 명령으로 듀티 50~90% 변경 / 10kHz PWM

세부수행내용 (계속)

□ ISR별 이벤트 전달

인터럽트	ISR 처리
EXTI15_10 (PC13)	Pending을 먼저 해제한 뒤 누름·뗌을 판별해 이벤트 큐에 저장
TIM2 / TIM4	UIF 해제 후 만료 플래그 TIM2_Expired / TIM4_Expired 설정
USART2 RX	SR→DR 순서로 읽어 오버런 해제, '1'~'9'이면 듀티 50~90% 선택 후 플래그, 그 외는 플래그만(경고는 메인)

TIM5는 상향 카운트 PWM mode 1, ARR 99(10kHz)로 한 번 설정하고, 속도 명령에는 CCR1·CCR2 = duty만 갱신합니다(핀 High 비율 = duty%).

관련 코드: exception.c (EXTI15_10, TIM2, TIM4, USART2 IRQHandler), timer.c, key.c(버튼 이벤트 큐)

□ 모터 상태 전이

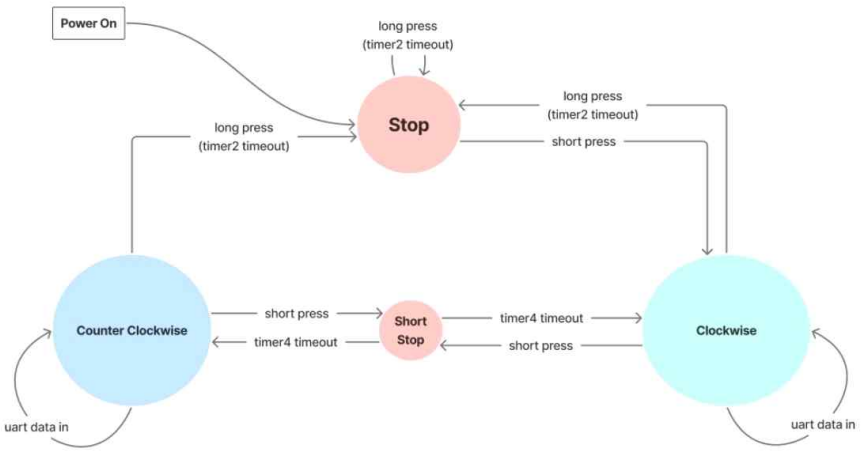


그림 5. 모터 상태도. 짧은 누름은 SHORT_STOP을 거쳐 방향을 바꾸고, 길게 누름(TIM2 만료)은 어느 상태에서든 STOP으로 전환합니다.

사후 점검에서 3초 무렵에 떼면 방향 전환과 정지가 함께 실행되고 정지 뒤 모터가 다시 도는 경계 조건을 발견해, 만료를 뗌보다 먼저 처리하고 정지할 때 TIM4를 취소하도록 수정했습니다.

□ 동작 확인

실물 보드에서 UART 속도 변경 후 정지 동작을 확인했습니다. 수정 후 3초 경계 시험: 3회 중 3회 정상

■ STM32 WS2812B Timer·DMA 제어 (개인 실습 | 2026.07.29)

□ 비교값 배열을 DMA로 전송

CPU 색상 버퍼 → DMA1 Stream1 → TIM2 CCR3 → PB10 → LED DIN

4개 LED의 GRB 데이터를 MSB부터 96개 비교값으로 변환하고, 뒤에 Low(CCR3 = 0) 주기 2개를 붙였습니다. DMA가 비교 레지스터를 갱신해 비트마다 CPU가 출력을 조작하지 않도록 했으며, 전송 전에는 PB10을 Low로 유지해 Reset 구간을 만든 뒤 타이머 기능으로 전환했습니다.

세부수행내용 (계속)

설정	설계값
TIM2 96MHz · PSC 0 · ARR 119 · 상향 카운트	비트 주기 1.25μs
CCR3 32 / 82	High 약 0.33μs / 0.85μs
TIM4 One-pulse (PSC 95, 1카운트 1μs)	Reset 300μs 후 PWM·DMA 시작

□ DMA·타이머 레지스터 설정

레지스터	설정 내용
DMA1 Stream1 CR	채널 3(TIM2_CH3), 메모리→주변장치, 메모리 주소 증가, 32비트 전송, 전송 완료·오류 인터럽트
PAR / M0AR / NDTR	TIM2→CCR3 / 비교값 버퍼 / 98개 (데이터 96 + Low 2)
TIM2 DIER bit 11	CC3 DMA 요청 허용 (전송 완료 후 해제)

전송 완료(TCIF1) 인터럽트에서 플래그를 지우고 CC3 DMA 요청을 끈 뒤 is_dma_busy를 0으로 바꿔 다음 전송을 허용합니다. 프리로드로 값이 한 주기 늦게 출력되므로, 뒤의 Low 주기 덕분에 완료 시점에는 마지막 비트가 이미 끝나 있습니다.

□ 데이터시트 규격과 설계값 대조

항목	설계값 (레지스터 계산)	WS2812B 규격	판정
비트 주기	1.250μs	1.25μs ± 600ns	범위 내
T0H / T0L	0.333μs / 0.917μs	0.40μs / 0.85μs ± 150ns	범위 내
T1H / T1L	0.854μs / 0.396μs	0.80μs / 0.45μs ± 150ns	범위 내
Reset	300μs	구형 50μs·V5 280μs 이상	범위 내

※ Reset 300μs는 구형과 신형(V5 계열) WS2812B의 조건을 모두 만족합니다.

□ 문제 해결

현상 1	Reset 대기가 설정 시간보다 짧게 끝남
원인 · 해결	설정 적용을 위한 UG 비트가 UIF를 세트해 인터럽트 허용 즉시 만료로 처리됨 → UIF와 NVIC Pending을 지운 뒤 인터럽트 허용
현상 2	정상 데이터 앞에 불필요한 High·Low 구간이 생길 수 있음
원인 · 해결	DMA 설정 전에 TIM2가 시작되면 초기 CCR 값이 출력됨 → 버퍼 생성 → TIM2 설정 → DMA 시작 → TIM2 시작 순서로 고정

□ 결과와 확인 범위

WS2812B 4개의 실제 점등을 확인했습니다. 비트 주기와 펄스 폭은 레지스터 설정에 따른 설계값이며, 위 표와 같이 데이터시트 규격과 대조했습니다. 사후 점검에서 하향 카운트로 High가 1카운트 길던 설정과 마지막 비트가 잘릴 수 있던 문제를 찾아 수정했습니다.

프로젝트 기술서

작 성 자
곽혜민

3	프로젝트명 : AVR 기반 RTC·LCD 계산기 통합 시스템
수행기간	2026.07.01 ~ 2026.07.03
담당역할	2인 프로젝트 · LCD 드라이버와 계산기 로직 담당 (DS1307 I2C 통신은 팀원 담당) 담당 범위: LCD 초기화·출력 드라이버, 수식 입력·편집 화면, 스택 기반 계산 로직
수행목표	LCD1602를 4비트 방식으로 직접 제어하고, 키패드 입력을 괄호 연산까지 처리하는 계산기로 구현 DS1307 RTC의 시간·날짜를 LCD1602에 표시하고, 4×4 키패드와 버튼으로 시간 설정과 괄호가 포함된 사칙연산 계산기를 사용하는 시스템입니다.
사용 기술	C · ATmega128A · 16MHz · Atmel Studio 7 / AVR GCC 주요 구성: LCD1602 · DS1307 RTC · 4×4 키패드 · 버튼 5개 · 피에조 부저
온라인 포트폴리오	https://hyehye12.github.io/projects/avr-rtc-calculator

세부수행내용

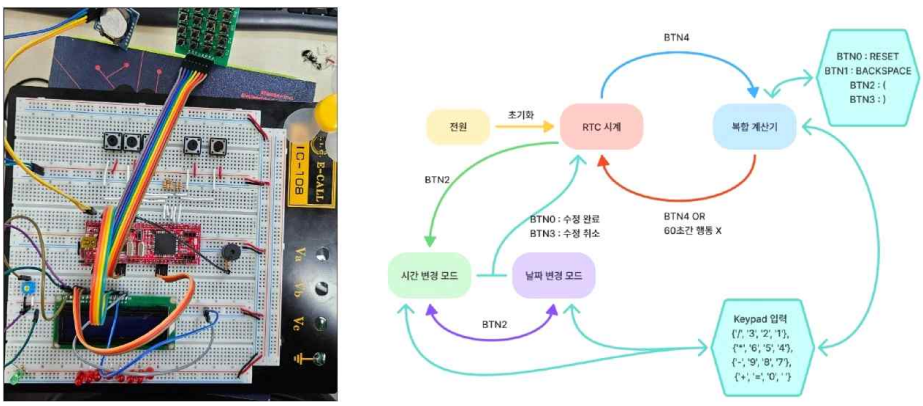


그림 6. 브레드보드 구성(왼쪽)과 모드 상태도(오른쪽). RTC 시계에서 BTN으로 시간·날짜 설정과 계산기 모드로 전환합니다.

■ LCD1602 4비트 제어

처리	구현 내용
초기화	전원 후 15ms 대기 → 0x30을 3회 전송(4.1ms·100µs·100µs 간격)해 8비트 상태로 맞춤 → 0x20으로 4비트 전환 → 0x28(2줄) · 0x08 · 화면 지우기 · 0x06 · 0x0C
데이터 전송	상위 니블 → E 펄스 → 하위 니블 → E 펄스, RS로 명령과 문자를 구분하고 R/W는 Low로 고정

세부수행내용 (계속)

처리	구현 내용
커서 이동	0x80 + (행 × 0x40) + 열, 1행 시작 주소 0x00 · 2행 시작 주소 0x40

문제	LCD에 글자가 출력되지 않거나 깨짐
원인 · 해결	4비트 모드로 제대로 초기화되지 않음 → 데이터시트의 초기화 순서대로 8비트 명령을 세 번 보내 상태를 맞춘 뒤 4비트 모드로 설정

```
void lcd_cmd_exe_4bit(uint8_t cmd) {
    LCD_CONTROL_PORT &= ~(1 << LCD_RS_PIN);   // 명령: RS = 0
    LCD_DATA_PORT = cmd & 0xf0;              // 상위 니블
    lcd_purse_enable();
    LCD_DATA_PORT = (cmd << 4) & 0xf0;        // 하위 니블
    lcd_purse_enable();
    _delay_ms(2);
}
```

데이터 핀은 포트 상위 4비트(DDR 0xF0)만 사용하고, 하위 니블은 4비트 왼쪽으로 옮겨 같은 핀으로 보냅니다. 초기화 단계의 0x30 3회 전송은 상위 니블만 보내는 lcd_cmd_exe_4bit_once로 처리했습니다.

■ 스택 기반 계산기와 수식 입력

□ 숫자 스택과 연산자 스택으로 중위식 계산

입력한 수식을 후위식으로 바꾸지 않고, 숫자 스택과 연산자 스택 두 개로 바로 계산했습니다. 연산자가 들어올 때 스택 위의 연산자와 우선순위를 비교해 먼저 계산할 것을 처리하므로, 곱셈·나눗셈 우선과 왼쪽부터 계산하는 순서가 함께 지켜집니다.

입력	처리
숫자	자릿수를 누적하고 take_num 플래그로 숫자 입력 중임을 표시, 연산자나 괄호가 오면 숫자 스택에 저장
+ − × ÷	스택 위 연산자의 우선순위가 같거나 높으면 먼저 계산해 결과를 숫자 스택에 넣고, 그 뒤 현재 연산자를 저장
(/)	여는 괄호는 우선순위 0으로 저장, 닫는 괄호에서는 여는 괄호까지 반복 계산한 뒤 제거
=	남은 연산을 모두 계산하고 결과를 소수 둘째 자리까지 표시 (1천만 이상은 지수 표기, 오류는 문구 표시)

```
int priority(char op) {
    switch (op) {
    case '(':           return 0;
    case '+': case '-': return 1;
    case '*': case '/': return 2;
    }
    return -1;
}
```

※ 우선순위: (= 0, + − = 1, × ÷ = 2. 여는 괄호를 가장 낮게 두어, 괄호 안에서는 바깥 연산자를 먼저 꺼내 계산하지 않습니다.

세부수행내용 (계속)

□ 수식 입력과 편집

입력	동작
BTN0 / BTN1	수식 전체 지우기 / 한 글자 지우기 (지운 뒤 다시 출력하고 커서 위치 복원)
BTN2 / BTN3	여는 괄호 / 닫는 괄호 입력 (키패드에 괄호가 없어 버튼에 배정)
16자 초과 / 32자	2행으로 이어서 입력, 32자(두 줄)에서 입력 제한, 지울 때는 두 줄을 다시 그림
키 입력 보관	키패드 ISR(20ms 주기)이 놓은 키를 8칸 큐에 넣고 메인이 꺼내 처리 (LCD 갱신 중에도 입력 유실 방지)
60초 미입력	계산기를 빠져나와 시계 모드로 복귀

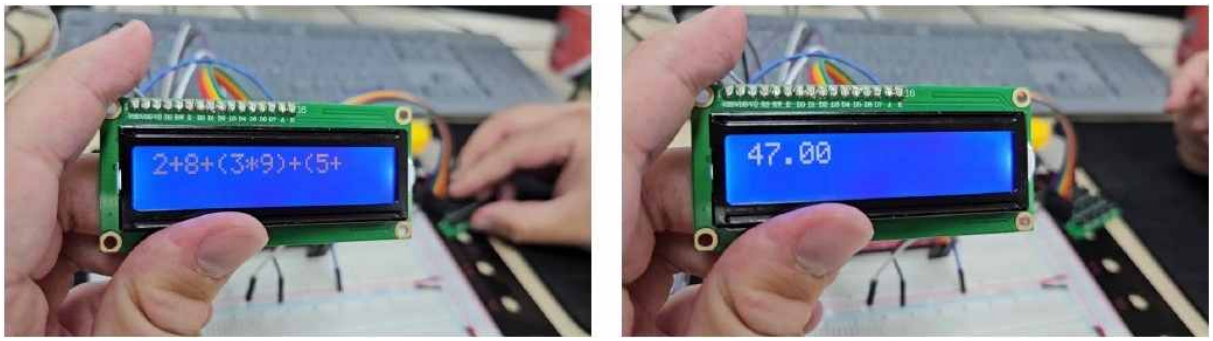


그림 7. 시연 영상 캡처. 괄호가 포함된 수식 2+8+(3*9)+(5+... 입력(왼쪽)과 = 입력 후 결과 47.00 표시(오른쪽)

□ 괄호 처리 보완과 추가 검증

현상	괄호 안에 연산자가 두 개 이상이면 결과가 0으로 표시됨 (예: (2+3*4), 2*(3+4*5))
원인	닫는 괄호에서 조건문으로 한 번만 연산해, 남은 연산자와 여는 괄호가 스택에 남음
해결	여는 괄호가 나올 때까지 반복해서 계산하도록 조건문을 반복문으로 수정
확인	PC 실행 시험(AVR과 같은 32비트 float)에서 수식 32개 통과: 정상 수식((2+3*4)=14, 2*(3+4*5)=46 등), 16비트 초과(40000+1), 단항 마이너스, 오류 13종. 보드에서도 2*(3+4*5) 입력 시 46.00 표시 확인
사후 보완	0으로 나누기·잘못된 수식·괄호 불일치를 오류 문구로 표시, 숫자는 7자리까지 (int16 → int32, float 정밀도 고려)

```
// 수정 전: 닫는 괄호에서 한 번만 계산
if (!op_empty(&op_stack) && op_peep(&op_stack) != '(')
// 수정 후: 여는 괄호가 나올 때까지 반복 계산
while (op_peek(&ops) != '(')
    if ((status = reduce_once(&nums, &ops)) != CALC_OK) return status;
```

□ 결과와 배운 점

LCD를 데이터시트의 초기화 순서대로 직접 4비트 제어하고, 두 스택으로 괄호와 우선순위가 있는 수식을 계산했습니다. 반례 수식을 직접 만들어 시험하며 조건문과 반복문의 차이 같은 경계 조건을 확인하는 습관을 얻었습니다.

프로젝트 기술서

작 성 자
곽혜민

4	프로젝트명 : Jetson 기반 스마트 선풍기
수행기간	2026.10.08 ~ 2026.10.13
담당역할	팀 프로젝트 · BMP180 센서 측정 및 16×2 LCD 출력 담당 (모터 커널 드라이버와 엔코더 구현은 팀원 담당) 담당 기능: I2C 센서 통신 · 원시값 보정 · 측정 데이터 검증 · 상태 표시 · 표시 작업 분리
수행목표	Linux 환경에서 온도·기압 측정값과 팬 상태를 연결하는 센서·표시 부분 구현 BMP180 보정값 처리와 LCD 갱신을 담당했습니다.
사용 기술	C · Linux · Jetson Orin Nano · BMP180 · 16×2 LCD(I2C) · pthread
온라인 포트폴리오	https://hyehye12.github.io/projects/smart-fan

세부수행내용

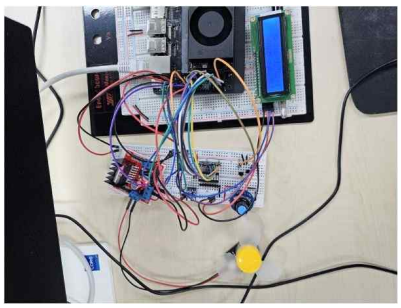


그림 8. 하드웨어 구성. Jetson Orin Nano, BMP180, 16×2 LCD, 모터 드라이버와 팬을 연결한 모습입니다.

■ 센서 데이터가 표시되기까지

BMP180 → I2C 읽기 및 보정 → 센서 프로세스 → 파이프
메인 상태 갱신 → LCD 작업 스레드 → I2C LCD 출력

시스템은 센서 데이터를 별도 프로세스에서 전달받습니다. 파이프 수신부는 부분 수신 데이터를 모아 프레임을 완성하고 매직값, 순서 번호와 시각을 검사합니다. 검사에 실패하면 파이프를 닫고 센서 오류로 표시하며, 측정값이 3초 넘게 갱신되지 않으면 자동 모드에서 팬을 정지합니다.

검사 항목	거부 조건
부분 수신	프레임 크기만큼 모일 때까지 누적, 중간에 끊기면 EPROTO
매직값 · 예약 필드	SENSOR_MAGIC 불일치 또는 예약 필드가 0이 아님
순서 번호	직전 번호 + 1이 아님 (누락·중복 프레임)
시각	현재 시각보다 100ms 넘게 앞섬

세부수행내용 (계속)

검사 항목	거부 조건
측정값 범위	성공 프레임에서 온도 -40~85°C, 기압 30,000~110,000Pa 범위 밖

□ BMP180 초기화와 측정

단계	처리 내용
장치 확인	I2C 주소 0x77, ID 레지스터 0xD0에서 0x55 확인
보정값 읽기	0xAA부터 22바이트 읽기, 부호와 바이트 순서에 맞춰 변환
온도·기압 측정	온도 명령 0x2E, 기압 OSS 3 설정으로 원시값 읽기
물리량 변환	보정식 적용 후 온도 m°C, 기압 Pa 단위로 전달
유효성 검사	온도 -40~85°C, 기압 30,000~110,000Pa 범위 확인

```
static uint16_t be16(const uint8_t *p) { return (uint16_t)(p[0] << 8) | p[1]; }
.ac4 = be16(bytes + 6),      /* AC4~AC6: 부호 없는 값 */
.ac1 = signed16(bytes),      /* 나머지 8개: 부호 있는 값 */
if (zero || erased || !c->ac4 || !c->ac5) { errno = EINVAL; return -1; }
```

22바이트를 빅엔디언 16비트로 묶고, AC4~AC6은 부호 없는 값, AC1~AC3·B1·B2·MB·MC·MD는 부호 있는 값으로 변환했습니다. 22바이트가 모두 0x00이거나 0xFF(지워진 EEPROM)이면, 또는 AC4·AC5가 0이면 잘못된 보정값으로 보고 EINVAL을 반환합니다.

보정식은 데이터시트 3.5절의 예제 값(원시값 27898, 23843 → 15.0°C, 69,964Pa)과 결과가 일치하는지 단위 시험으로 확인했습니다.

관련 코드: app/bmp180.c · 시험 코드: tests/test_bmp180.c

□ 담당 범위의 결과

센서의 레지스터 값이 화면에 표시되기까지 초기화, 보정, 유효성 검사와 출력 단계를 연결했습니다. 단순 원시값 읽기에서 끝내지 않고, 데이터 단위와 오류 처리 기준을 정리했습니다.

■ LCD 갱신과 오류 처리

□ 표시 작업을 제어 처리에서 분리

LCD 갱신은 별도 작업 스레드에서 수행했습니다. 상태 변경 통지를 받으면 표시할 값만 mutex 안에서 복사하고, mutex를 해제한 뒤 I2C로 출력해 제어 쪽이 LCD 전송을 기다리지 않도록 했습니다. 이전 화면과 같은 줄은 다시 전송하지 않습니다.

처리 대상	적용한 방법
LCD 1바이트 전송	상·하위 니블로 나누고 Enable Low-High-Low를 포함해 6바이트 전송
문자열 길이 감소	한 줄을 16칸으로 채워 이전 문자열의 남은 문자 제거
중복 갱신	이전 줄과 비교해 같은 내용의 I2C 전송 생략
통신 오류	부분 전송 오류를 반환하고 메인에서 경고 처리

세부수행내용 (계속)

□ LCD 표시 규칙

표시 위치·조건	표시 내용
1행	FAN ON/OFF · AUTO/MANUAL
2행 — 센서 사용·온도값 유효	온도 소수 첫째 자리 · 속도 S:n/5
2행 — 센서 사용·온도값 무효	T:ERR · 속도 S:n/5
2행 — 센서 미사용	속도 SPEED:n/5 · LED 개수



그림 9. LCD 표시(왼쪽: FAN ON, MANUAL, 20.3°C, 속도 3/5)와 시연 영상 캡처(오른쪽: 센서를 손으로 감싸 온도를 올리자 로그의 측정 온도가 오르고 목표 풍속이 바뀌는 장면)

□ I2C 설정과 부분 전송 오류 처리

LCD와 센서의 기본 I2C 버스는 /dev/i2c-7이며 실행 옵션으로 각각 변경할 수 있습니다. LCD 부분 전송 실패 시 이미 Enable 펄스가 전달됐을 가능성을 고려해 남은 바이트를 재전송하지 않고 오류를 반환합니다. 이를 통해 불완전한 재전송으로 4비트 데이터 순서가 어긋나는 상황을 피하도록 했습니다.

```
ssize_t count = write(fd, bytes, length);
if (count < 0)
    return -1;
if ((size_t)count != length) {
    errno = EIO;    /* 부분 전송도 실패로 처리 */
    return -1;      /* 남은 바이트는 재전송하지 않음 */
}
```

※ 1바이트는 6바이트로 인코딩해 한 번에 쓰고, 화면 지우기·커서 복귀 명령 뒤에는 2ms, 그 외에는 60μs를 기다립니다.

□ 장치 오류의 영향 범위

LCD 오류는 상위 처리로 전달하되 모터 제어는 계속 수행하도록 구성했습니다. 표시 장치의 실패가 전체 제어 중단으로 이어지지 않도록 오류 전달과 제어 지속 여부를 구분했습니다. 다만 LCD 스레드는 한 번 실패하면 종료되므로 재시작 처리가 개선 과제입니다.

□ 결과와 배운 점

I2C 장치마다 다른 데이터 형식과 출력 순서를 다루고, 스레드 사이에 공유되는 상태를 보호하는 경험을 했습니다. 화면 갱신 주기와 제어 로직을 분리하고 오류가 발생한 장치의 영향을 제한하는 구조를 구현했습니다.

프로젝트 기술서

작 성 자
곽혜민

5	프로젝트명 : Basys3 기반 I2C 센서 모니터링 시스템
수행기간	2026.11.03 ~ 2026.11.05
담당역할	2인 프로젝트 · 슬레이브 보드 RTL 전체와 Qt 모니터 담당 (마스터 보드의 I2C 제어와 OLED 출력은 팀원 담당)
수행목표	센서값과 정상 여부를 한 프레임으로 정의해 I2C·UART에 같은 의미로 전달 두 대의 Basys3 중 슬레이브 보드가 DHT11 온습도와 HC-SR04 거리를 측정합니다. 마스터는 I2C로 읽어 OLED에, PC는 UART로 받아 Qt 모니터에 표시합니다.
사용 기술	Verilog · Vivado 2020.2 · Basys 3 (Artix-7, 100MHz) · C++ · Qt 5 주요 구성: DHT11 · HC-SR04 · 4자리 FND · USB-UART (마스터: SSD1306 OLED)
온라인 포트폴리오	https://hyehye12.github.io/projects/basys3-i2c-sensor-monitor

세부수행내용

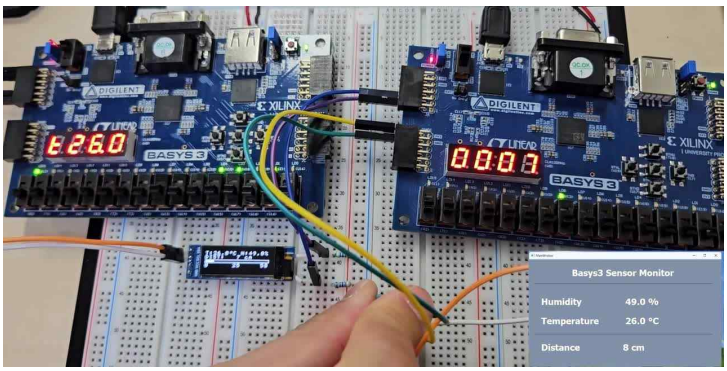


그림 10. 시연 영상 캡처. 왼쪽 마스터 보드(FND·OLED)와 오른쪽 슬레이브 보드(FND 거리 cm), 오른쪽 아래 Qt 모니터가 같은 측정값을 표시합니다.

■ 측정부터 전송까지

구성	설계 내용
DHT11	40비트 수신, HIGH 폭 50µs 이상을 1로 판별, 체크섬 일치 시 갱신
HC-SR04	100ms마다 TRIG 10µs, ECHO 폭(µs)으로 2~255cm 측정
I2C 슬레이브	주소 0x42, 상태·온습도·거리 6바이트 응답
UART · FND	1초마다 AA 55 + 6바이트 전송(9600bps), FND에 거리 표시

세부수행내용 (계속)

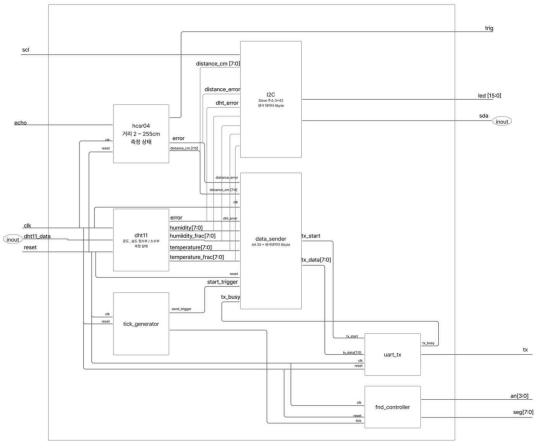


그림 11. 슬레이브 블록 다이어그램. 센서 모듈의 측정값과 오류 신호가 I2C 모듈과 data_sender(UART)로 함께 전달됩니다.

□ 데이터 프레임과 상태 헤더

레지스터	내용	비고
0x00	상태 헤더	101000 + 초음파·온습도 정상 비트
0x01 ~ 0x04	온도·습도 정수/소수	DHT11 40비트 프레임에서 추출
0x05	거리(cm)	마지막 바이트, 마스터가 NACK

측정에 실패하면 마지막 정상값을 유지하고 해당 상태 비트만 0으로 바꿉니다. 상위 6비트 서명으로 통신 이상과 센서 이상을 구분하며, A3은 모두 정상, A2는 온습도 오류, A1은 초음파 오류입니다. UART는 같은 6바이트 앞에 AA 55를 붙여 보냅니다.

■ I2C 슬레이브 설계

□ 버스 신호 동기화와 응답 FSM

SCL·SDA를 2단 FF로 동기화하고 직전 값과 비교해 SCL 에지와 START·STOP을 검출했습니다. SDA는 0일 때만 구동하고 1은 선을 놓아 풀업이 올리는 오픈 드레인 방식입니다. 주소가 일치한 순간의 측정값을 저장해 6바이트를 읽는 동안 값이 섞이지 않게 했습니다.

```
wire start_seen = scl_high &&  sda_prev && !sda_in;  // START
wire stop_seen  = scl_high && !sda_prev &&  sda_in;  // STOP
assign sda = drive_low ? 1'b0 : 1'bz;  // 오픈 드레인
if (received[7:1] == ADDRESS) begin    // 주소 0x42 일치 시 값 저장
    state <= ACK_BEGIN;
    saved_temp <= temperature; saved_distance <= distance_cm;
end
MASTER_ACK: if (scl_up) state <= sda_in ? IDLE : NEXT_TX; // NACK면 종료
```

주소 수신 → ACK → 레지스터 값 송신(MSB부터) → 마스터 ACK: 다음 레지스터 / NACK: SDA 해제

통합 시험에서 마스터가 마지막 바이트에도 ACK를 보내, 슬레이브가 다음 바이트의 MSB(0)로 SDA를 잡아 STOP이 성립하지 않고 버스가 멈췄습니다. 로직 분석기로 6번째 바이트 뒤 SDA가 LOW로 고정된 것을 확인했고, 팀원이 마지막 바이트만 NACK하도록 마스터를 수정해 해결했습니다.

세부수행내용 (계속)

□ 문제 해결: 거리 환산 나눗셈의 타이밍 위반

현상	Vivado 구현 단계에서 타이밍 조건 미충족 경고 발생 (WNS −1.555ns)
원인	distance_cm <= (count_us + 29) / 58의 나눗셈 조합 경로가 100MHz(10ns) 설정 시간을 넘김
해결	CALCULATE 상태를 추가해 1μs마다 58을 빼고 몫을 1씩 늘리는 순차 연산으로 변경 (+29로 반올림 유지)
결과	WNS +2.472ns로 타이밍 충족, 반복은 최대 255회(약 0.26ms)로 100ms 측정 주기 안에서 완료

□ 통합 테스트벤치와 실물 확인

테스트벤치에서 DHT11 40비트 프레임(61.7%, 25.4°C)과 580μs ECHO(10cm)를 모사하고, I2C 읽기 동작으로 받은 6바이트를 기대값 A3 19 04 3D 07 0A와 비교했습니다. UART 출력도 LSB부터 복원해 같은 값인지 확인하며, 모두 일치하면 PASS: I2C and UART를 출력합니다.

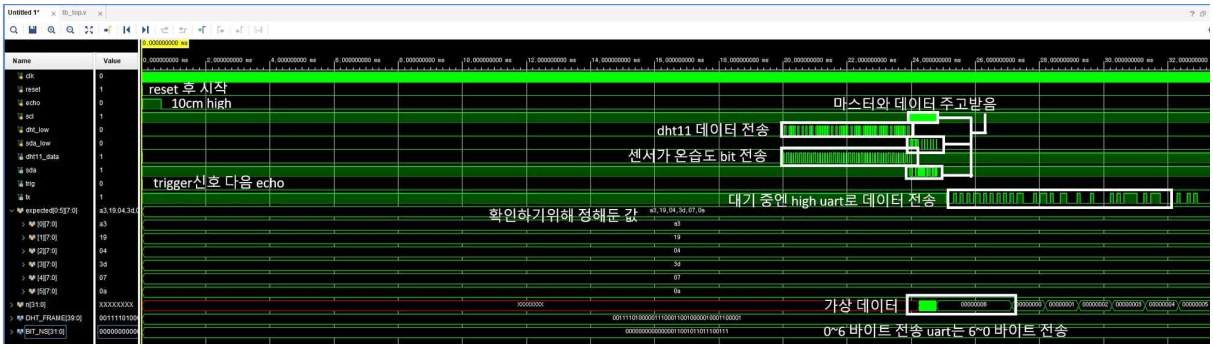


그림 12. 통합 테스트벤치 파형. reset → TRIG/ECHO(10cm) → DHT11 40비트 수신 → I2C 6바이트 응답 → UART 전송

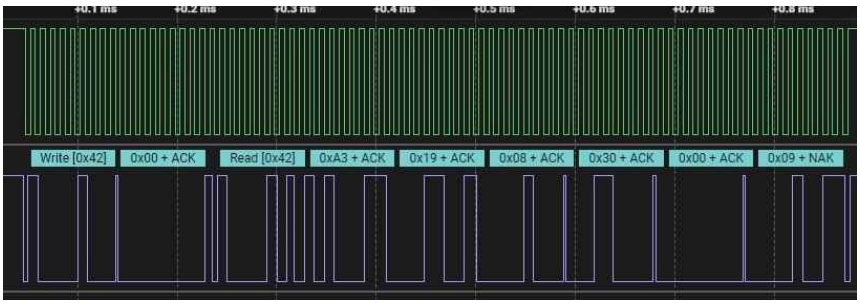


그림 13. 실측 I2C 파형(로직 분석기). 슬레이브 응답 A3 19 08 30 00 09가 마스터 OLED의 25.8°C, 48.0%, 9cm와 일치하고, 마지막 바이트 NAK 후 STOP으로 종료됩니다.

시험 코드: sim/tb_top.v · 관련 코드: rtl/slave/i2c.v, hcsr04.v, dht11.v, data_sender.v, qt/mainwindow.cpp

□ 결과와 배운 점

센서 측정, 상태 판정, I2C·UART 전송을 슬레이브 FPGA 하나에 통합하고, 마스터와 PC가 같은 데이터를 같은 의미로 해석하도록 프레임 규약을 정했습니다. 타이밍 위반을 해결하며 FPGA에서는 계산 결과뿐 아니라 계산에 걸리는 시간도 설계 조건이라는 점을 배웠습니다.

프로젝트 기술서

작 성 자
곽혜민

6	프로젝트명 : STM32와 Qt 차량 제어 모니터링
수행기간	2026.08.14 ~ 2026.08.19
담당역할	4인 프로젝트 · Qt GUI 전체 담당 화면 구성, 사용자 입력, 카메라 스레드와 세션 기록 담당 (MCU 펌웨어와 SerialWorker는 팀원 구현과 연동)
수행목표	Qt GUI에서 차량 제어 명령을 보내고 온습도·카메라 영상을 표시
사용 기술	C++ · Qt 5 Widgets · SerialPort · Multimedia 연동 대상: STM32 · UART 115200bps, 모터·조향·방향지시등 제어 입력, 센서와 영상 표시
온라인 포트폴리오	https://hyehye12.github.io/projects/vehicle

세부수행내용

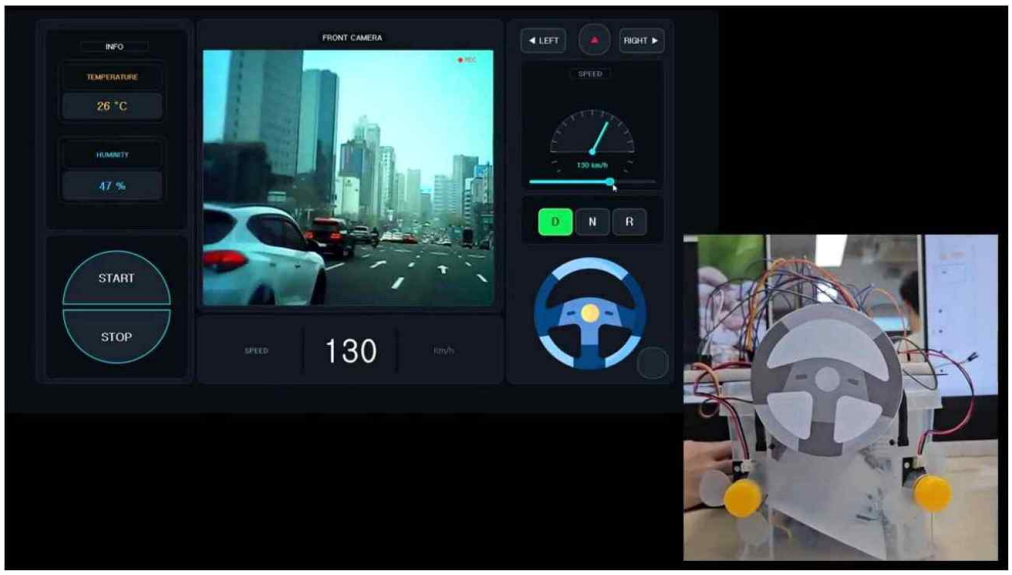


그림 14. 시연 영상의 GUI와 차량 장치. 게이지의 130은 GUI가 보낸 속도 명령값이고, REC는 화면 표시 요소입니다.

■ 연결 상태와 사용자 입력

연결 성공 전에는 제어 입력을 비활성화하고, 연결 후 기어·속도·조향 입력을 사용할 수 있도록 했습니다. 속도는 슬라이더를 놓을 때 전송하며 QPainter 게이지와 핸들 이미지 회전으로 입력 상태를 표시했습니다.

세부수행내용 (계속)

□ 시스템 연결

Qt GUI → SerialWorker → UART → STM32 → 모터·서보·LED
DHT11 → STM32 → UART → GUI 온습도 표시 / PC 카메라 → GUI 영상

■ 명령 처리와 카메라 종료 관리

□ 명령과 수신 데이터 형식

구분	형식	GUI 동작
기어	D+F/N/R	전진·중립·후진 선택 전달
속도	S + 0~200	슬라이더 해제 시 전송
조향	A + -90~90	조향 입력과 핸들 이미지 연동
방향지시등	F+L/R/E	좌·우 버튼 토글 시 F+L/R, 비상등 버튼 F+E
온습도 수신	$온도,습도 또는 ERR	값 표시 또는 오류 처리

송신 명령은 줄바꿈으로 구분합니다. D와 R 사이를 직접 바꿀 때 슬라이더 속도 값이 0이 아니면 변경을 막고 경고 후 이전 선택으로 되돌렸습니다. N을 거치는 전환은 막지 않는 한계가 있습니다.

□ 카메라 작업과 종료 순서

CameraThread에서 카메라를 초기화하고 이벤트 루프를 실행했습니다. 프레임은 QImage 변환과 상하 반전 후 시그널로 UI에 전달합니다. 여러 카메라가 있으면 목록의 두 번째 장치를 선택하도록 구성했습니다.

종료 요청 → quit 및 wait → 카메라 정지 → 관련 객체 해제

화면을 닫을 때도 카메라 작업이 남지 않도록 스레드 종료를 기다린 뒤 자원을 정리했습니다. 프레임은 convertToFormat()으로 복사본을 만든 뒤 unmap()하므로, UI로 전달되는 이미지는 매핑된 영상 메모리를 참조하지 않습니다.

□ 세션 기록의 의미

기록 항목	계산 또는 수집 기준
세션 시간	연결 시작부터 종료까지의 시간
최고·평균 속도 명령값	전송한 슬라이더 값의 최댓값과 산술평균
방향지시등 작동 횟수	좌·우 버튼 작동 기록 (실제 회전 횟수와 구분)
온습도	수신한 최근 값

□ 결과와 배운 점

연결, 입력, 수신, 영상 표시와 종료를 하나의 화면 흐름으로 구성했습니다. 장치의 실제 측정값과 사용자가 보낸 명령값을 구분해 기록의 의미를 명확히 했습니다.

프로젝트 기술서

작 성 자
곽혜민

7	프로젝트명 : Jetson 손동작 인식 가위바위보
수행기간	2026.08.28 ~ 2026.08.31
담당역할	개인 프로젝트 · 데이터 수집부터 학습·변환·게임 구현까지 전체 담당 직접 촬영·라벨링, 모델 학습·변환, 추론 연동, 영상 처리와 게임 상태 관리
수행목표	직접 학습한 손동작 인식 모델을 Jetson Orin Nano에서 실시간 추론해 가위바위보 게임으로 구현 카메라 입력부터 검출 후처리, 손동작 확정과 PyQt5 게임 화면까지 연결했습니다.
사용 기술	Python · YOLO11n · ONNX · TensorRT · OpenCV · PyQt5 · Jetson Orin Nano
온라인 포트폴리오	https://hyehye12.github.io/projects/jetson-rps-game

세부수행내용

■ 모듈 구성

파일	역할
rps_game.py	검출·후처리, 플레이어 선택, 손동작 확정, 승패 판정 함수
rps_game_gui.py	모드 전환, 라운드 진행, 화면 표시
trt_module.py	TensorRT 엔진 로딩과 추론 실행

■ 데이터 준비와 모델 배포 과정

단계	수행 내용
직접 촬영	가위·바위·보 손동작 사진 총 약 320장 촬영
라벨링	LabelImg로 손 영역과 클래스 지정, XML 라벨을 YOLO 형식으로 변환
Colab 학습	YOLO11n 사전학습 모델 미세조정. 학습 설정: 80 epochs, batch 32, 입력 크기 320×320
ONNX 내보내기	Colab에서 best.pt를 ONNX로 내보내기. 입력 크기 320×320, opset 18, simplify=True
TensorRT 변환	Jetson Orin Nano의 터미널에서 명령어로 ONNX를 TensorRT 엔진으로 변환하고 게임에 연동

세부수행내용 (계속)

□ 카메라 프레임 처리

카메라 → RGB 변환 → letterbox → 정규화 및 NCHW 배열
TensorRT 추론 → 클래스별 NMS → 원본 좌표 복원 → 손동작 후보

영상 비율을 유지한 채 letterbox 패딩값 114를 적용하고 픽셀값을 255로 나눠 정규화했습니다. 신뢰도 0.50, NMS IoU 0.45 기준으로 검출 결과를 정리하고, 패딩과 배율을 역적용해 손 영역 좌표를 원본 화면으로 복원했습니다.

※ 카메라는 640×480으로 요청하며, 실제 해상도는 카메라 지원 범위에 따릅니다.

□ TensorRT 추론 실행

엔진을 역직렬화하고 실행 컨텍스트와 CUDA 스트림을 생성했습니다. 각 입출력 텐서에는 GPU에서 접근할 수 있도록 매핑한 페이지 고정 메모리를 할당하고 주소를 연결했습니다. 전처리한 입력을 버퍼에 복사한 뒤 execute_async_v3()로 추론을 실행하며, 스트림 동기화로 실행 완료를 기다린 후 출력 버퍼를 반환합니다.

□ 직접 맡은 범위

직접 촬영과 라벨링부터 Colab 학습·ONNX 내보내기, Jetson의 TensorRT 변환과 게임 연동까지 수행했습니다. 연속 프레임의 결과를 이용해 손동작을 확정하고, 인식 결과를 게임 상태 전환과 점수 처리에 연결했습니다.

■ 손동작 안정화와 게임 상태 관리

□ 한 프레임의 오검출이 즉시 점수가 되지 않도록 처리

최근 7개 결과를 저장하고 같은 손동작이 5회 이상 나타날 때만 확정했습니다. 조건을 만족하지 못하면 미확정으로 두어 점수에 반영하지 않습니다. 싱글 모드는 신뢰도가 가장 높은 손을 사용하고, 멀티 모드는 화면 중심을 기준으로 좌·우 플레이어를 나눴습니다.

구성	처리 기준
클래스	0 가위 / 1 바위 / 2 보
후보 선택	싱글 최고 신뢰도 / 멀티 좌·우 영역별 최고 신뢰도
확정 조건	최근 7개 중 동일 결과 5회 이상
게임 흐름	3초 카운트다운 → 손동작 판정 → 결과 1.5초 표시
종료 및 오류	카메라 자원 해제, 인식 불가 시 점수 미반영

```
def stable_gesture(history):
    values = [v for v in history if v is not None]
    if len(values) < STABLE_FRAMES - 2:
        return None
    gesture, count = Counter(values).most_common(1)[0]
    return gesture if count >= STABLE_FRAMES - 2 else None
```

세부수행내용 (계속)

STABLE_FRAMES = 7이므로 미검출(None)을 뺀 유효 검출이 5개 이상이고, 가장 많이 나온 손동작이 5회 이상일 때만 확정합니다. 판정은 카운트다운이 끝나는 순간 한 번 하며, 그 시점의 최근 최대 7개 결과를 봅니다.



그림 15. 실제 게임 화면. 손 영역과 클래스를 표시하고 플레이어·컴퓨터의 선택 및 승패를 함께 보여줍니다.

□ 승패 판정과 실행 오류 대응

```
def winner(left, right):
    if left == right:
        return DRAW
    return PLAYER_1_WINS if (left - right) % 3 == 1 else PLAYER_2_WINS
```

가위=0, 바위=1, 보=2로 정의하고, 두 값이 같으면 무승부, (left - right) % 3 == 1이면 왼쪽 승리로 판정했습니다. 카메라 열기·프레임 읽기·추론 오류가 발생하면 메시지를 표시하고 타이머와 카메라를 정리한 뒤 메뉴 화면으로 복귀합니다. OpenCV가 설정한 Qt 플러그인 경로와의 충돌을 방지하기 위해 PyQt5의 플러그인 경로를 명시했습니다.

□ 결과와 개선 방향

직접 준비한 데이터가 실제 장치의 입력과 게임 결과로 이어지는 전체 과정을 구현했습니다. 연속 결과를 이용한 확정 로직으로 단일 프레임의 변동을 게임 판정과 분리했습니다. 추론이 GUI 스레드에서 실행되어 화면 갱신 주기(QTimer 30ms 요청)가 추론 시간에 좌우되므로, 추론 스레드 분리와 FPS 측정을 다음 개선 과제로 두었습니다.

프로젝트 기술서

작 성 자
곽혜민

8	프로젝트명 : Board Guard PCB 검사 모니터링
수행기간	2026.09.02 ~ 2026.09.15
담당역할	4인 프로젝트 · UI·UX 및 검사 이력 DB 담당 웹 UI: HTML · CSS · JavaScript, 검사 상태·영상·통계 표시 서버 및 DB 연동: Flask · SQLite, 확정 결과 저장 및 이력 조회 팀원 구현과 연동: PatchCore 추론 및 컨베이어·분류 장치 제어
수행목표	PCB 검사·분류 시스템에서 검사 화면과 이력 저장·조회 기능 구현 팀원이 구현한 AI 판정과 장치 제어 결과를 받아 현재 상태, 정상·불량 집계와 저장 이력을 확인할 수 있도록 구성했습니다.
사용 기술	HTML · CSS · JavaScript · Flask · SQLite
온라인 포트폴리오	https://hyehye12.github.io/projects/board-guard

세부수행내용

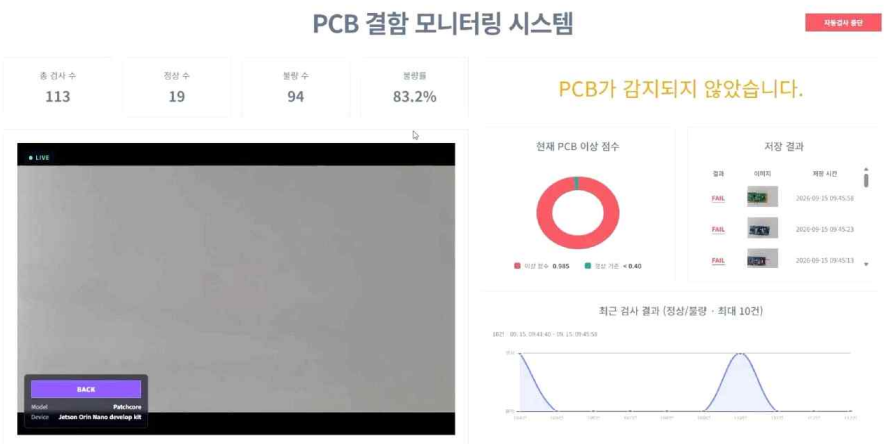


그림 16. 시연 영상의 대시보드. 검사 수·정상 수·불량 수, 현재 상태와 저장 결과를 한 화면에서 확인합니다. 불량률은 검사 집계 값입니다.

■ 현재 상태와 검사 이력의 구분

현재 상태는 500ms 간격으로 조회하고 영상 스트림을 별도로 표시했습니다. 검사 시작·중단과 앞·뒷면 선택은 서버 요청으로 전달하며, 확정된 검사 결과는 이력으로 저장해 현재 영상과 구분했습니다.

팀 AI·장치 제어 → Flask 상태 및 판정 결과 → 대시보드
확정 PASS·FAIL → SQLite → 검사 이력 조회 화면

세부수행내용 (계속)

■ 검사 결과 저장과 이력 조회

□ 확정 결과만 이력으로 저장

검사 상태가 PASS 또는 FAIL이고 검사 번호가 마지막 저장 번호보다 클 때 저장하도록 구성했습니다. 추론 후에도 검사 활성 상태와 앞·뒷면 선택이 유지되는지 확인해 상태 전환 중인 결과를 걸러내도록 했습니다.

저장 항목	목적
ID · UTC 검사 시각	이력 식별과 시간순 조회
앞·뒷면 · PASS/FAIL	검사 대상과 판정 상태 구분
점수 · 임계값	판정에 사용한 수치와 기준 기록
이미지 경로	검사 결과 이미지 연결
모델명·버전 · 추론 시간	결과가 생성된 실행 조건 기록
원인·결함 JSON	세부 판정 정보 보관

□ DB 접근과 화면 요청 관리

대상	구현 내용
동시 읽기·쓰기	SQLite WAL 및 busy_timeout 5초 설정
조회	시간·상태 인덱스, 매개변수 바인딩, 조건별 이력 조회
재시작	DB의 저장 이력으로 누적 집계 복원

□ 이력 조회와 응답 순서 관리

검사 이력은 PASS/FAIL 상태별로 조회하며, 기본 20건 단위 페이지와 앞면·뒷면 필터, 최근 최대 5건 조회를 지원합니다. API는 잘못된 조회 조건에 HTTP 400을 반환하고, 정상 조회 응답에는 캐시 저장 금지를 설정했습니다. 화면에서는 이전 요청을 취소하고 응답이 현재 요청에 해당하는지 다시 확인해 오래된 결과가 최신 필터 화면을 덮어쓰지 않도록 했습니다. 조회 실패 시에는 재시도 버튼을 표시합니다.

□ 결과와 개선 방향

검사 장치의 결과를 사용자가 확인하고 추적할 수 있는 UI와 저장 구조로 연결했습니다. AI 학습·판정 성능과 장치 제어는 팀원 담당이며, 제 기여는 검사 상태 표현, 확정 결과 저장과 이력 조회입니다.

마지막 저장 번호를 비교해 실행 중 같은 검사 결과가 반복 저장되는 것을 줄였습니다. 다만 추론 도중 중단과 재시작이 모두 일어나면 추론 후 확인 시점에는 상태가 같아 보여 이전 프레임 결과가 저장될 수 있어, 시작·중단마다 세대 번호를 두어 걸러내고 (세션 ID, 검사 번호)에 고유 제약을 두는 것이 개선 과제입니다.